

Exercise 2: Small Feature With Tests (Live) — 30 minutes

Scenario

Add one narrowly defined business rule to the inventory domain model:

An inventory item has a **reorder shortage**. If its `quantity` is below its `reorderLevel`, the shortage equals `reorderLevel - quantity`. If the quantity is at or above the reorder level, the shortage is `0`. The shortage must never be negative.

Examples the agent should satisfy: | quantity | reorderLevel | shortage | |-----:|-----:|-----:| | 2 | 10 | 8 | | 10 | 10 | 0 | | 25 | 10 | 0 | | 0 | 0 | 0 |

Workflow

1. Ask OpenCode to restate the requirement and identify the relevant files.
2. Ask for a short implementation plan.
3. Review the plan yourself.
4. Permit edits only to `InventoryItem.java` and its test file.
5. Run the targeted test.
6. Inspect the git diff.
7. Ask OpenCode to explain the changed lines.

Prompt template

implement this single change:

an inventory item has a reorder shortage. if quantity is below reorderLevel, the shortage equals reorderLevel - quantity. if quantity is at or above reorderLevel, the shortage is 0. the shortage must never be negative.

constraints:

- add the calculation only to src/main/java/com/atlas/inventory/InventoryItem.java and its unit test file, src/test/java/com/atlas/inventory/InventoryItemTest.java (create the test file if it does not exist yet).
- do not add dependencies.
- preserve the public api of InventoryItem (existing methods and the record's fields must not change).
- add tests for: quantity below reorderLevel, quantity equal to reorderLevel, quantity above reorderLevel, and both values at 0.
- do not refactor unrelated code.

first give me a plan in no more than five steps. wait for approval before editing.

After you approve the plan:

apply the plan now. make the smallest change possible.

then run only the InventoryItemTest and report:

- files changed,
- test command,
- test result,
- any remaining uncertainty.

Running the targeted test

```
mvn test -Dtest=InventoryItemTest
```

Confirm the agent used this command (or the equivalent `-pl / -am` variant) and not a full `mvn clean verify`, which would also exercise the repository and server tests unrelated to this change.

What "done" looks like

- A new method on `InventoryItem`, for example `reorderShortage()`, with no change to the existing fields, constructor, `withId`, or `toJson`.
- A new (or extended) `InventoryItemTest.java` covering the four cases above.
- `mvn test -Dtest=InventoryItemTest` passes.
- The diff touches exactly two files.

Java adaptation note

Attendees use the existing Maven test target shown above. If your fork of this exercise targets a Gradle or CMake project instead, substitute the equivalent single-test invocation for that build tool, and tell the agent that command explicitly rather than asking it to guess the build system.